Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Linear search with small and unsorted data

Today
Welcome to your next lesson: Linear search with small and unsorted data.

I'm setting up your lesson based on what you've already learned, what’s next for you, and the best way to build your skills step by step.

Let’s keep going — every bit adds up.

Imagine you have a short unsorted list of numbers on paper:

[7, 3, 10, 2, 8]

You want to know: “Is 10 in this list?”

You’d probably look at each number from left to right: check 7, then 3, then 10 — and stop when you find it. ☑

That “check one item after another until you find what you want (or hit the end)” has a name in algorithms: linear search.

Does that everyday idea of “scan through one by one” make sense so far?

yes though we have if 10 i list ad list.index(10) or however that wrks

Nice connection — if 10 in list and list.index(10) are both using the same idea under the hood: they still have to scan item by item. Linear search is just the explicit version of that. 🔍

I want to show it step by step in Python so you see exactly what’s happening.

Here’s the core pattern in words:

Start at the first element.
For each element in the list:
Compare it to the target.
If it matches → return its position.
If you finish the loop with no match → return something like -1 to say “not found”.
Let’s turn that into code together.

In the code editor, write a function linear_search(lst, target) that:

loops through the list,
returns the index of target if found,
returns -1 if not found.

Don’t worry if it’s not perfect on the first try — just give it a shot in the editor, hit Submit code, and I’ll review it.

Python

```
def linear_search(lst, target):
    found = False
    for item in lst:
        if item == target:
            return target
    return -1


my_list = [2, 4, 5, 6, 8, 10, 12, 15, 16]

result = linear_search(my_list, 11)
print(result)
```

Nice work getting that running, Ron 🎯

Right now your function is doing a linear search, but it’s returning the value (target) instead of the index/position in the list.

Step by step on your code:

Python

```
def linear_search(lst, target):
    found = False
    for item in lst:
        if item == target:
            return target   # <-- returns the value, not its index
    return -1
```

Two key things:

You don’t actually use found (so we can remove it).

To return the index, you want to loop with an index, like:

either for i in range(len(lst)): and use lst[i]
or for index, item in enumerate(lst):
Let’s do this in one small step:

In the editor, change your function so that it:

loops like this: for index, item in enumerate(lst):
when it finds the target, it returns index (not target)
keeps return -1 at the end if nothing is found

Update your code in the editor, hit Submit code again, and I’ll check it 👍

yeah i was thinking if not found loop then changed irection forgot to remove it

Gotcha, that totally tracks — that kind of “change direction mid-code” happens to everyone 😄

Let’s focus on the index version now, because that’s the classic linear search.

Visually, we want this behavior:

List	Target	What linear search should return
[2, 4, 5, 6]	5	2 (index of 5)
[2, 4, 5, 6]	2	0
[2, 4, 5, 6]	10	-1 (not found)

The key trick is to get both index and item while looping:

Python

```
for index, item in enumerate(lst):
    ...
```

Then you can return index when you find the match.

Can you update your function in the editor so it:

uses for index, item in enumerate(lst):
returns index when it finds the target
returns -1 at the end if not found

Run it with both a present value (like 10) and an absent one (11), then submit the code so I can take a look 👀

i must not be awake enough hmissed you wanted index specifially not just found or not

All good — you're awake enough 🙂 and you actually did the important part: you built the search correctly.

There are two common versions:

"Found or not" → return True / False
"Where is it?" → return the index or -1

You naturally wrote version 1 in your head; I asked for version 2 in code. Both are valid linear search.

Let's lock in version 2 now:

In the editor, adjust your function to something like this structure:

Python

```
def linear_search(lst, target):
    for index, item in enumerate(lst):
        if item == target:
            return index
    return -1
```

Try to write it yourself in the editor (don't copy-paste if you can avoid it), test with a found and not-found case, then hit Submit code and I'll review 💻

you want 0 or 1 returnd for 1st positon machine or human version

Great question 👌

Let's be super clear:

Python lists are 0-based.
So the first position has index 0, second is 1, third is 2, etc.

So for linear search in Python, we want the machine version:

first element → return 0
second element → return 1
…
not found → return -1

That’s the standard pattern you’ll see in most code.

Given that, can you now update your linear_search so:

searching [2, 4, 5, 6] for 2 returns 0
searching the same list for 6 returns 3
searching for 11 returns -1

Try it in the editor, then hit Submit code and I’ll check it 🎯

Python

```
def linear_search(lst, target):
    for index, item in enumerate(lst):
        if item == target:
            return index
    return -1


my_list = [2, 4, 5, 6, 8, 10, 12, 15, 16]

result = linear_search(my_list, 2)
print(result)
```

Nice, Ron — that’s a clean linear search that returns the correct index 🎯

For your test:

List: [2, 4, 5, 6, 8, 10, 12, 15, 16]
Target: 2
It returns: 0 ☑ (first position, 0-based)

So you’ve just implemented linear search correctly.

Quick next step: let’s talk about how many checks this can take.

Imagine you search this same list for 16:

How many elements might the code have to check in the worst case?
You don’t need to name it O(something) yet — just tell me:
would it check 1 item, all 9 items, or something in between in the worst case?
worst case al 9

Exactly — worst case it checks all 9 👍

That idea scales:

List length = n
Worst case for linear search = check all n items

In Big-O terms, we say its worst-case running time is O(n).

That just means: time grows in direct proportion to the number of items.

Does this match how you’re thinking about it, or does O(n) still feel a bit fuzzy?

yes single loops O(n)

Perfect: single simple loop over the list → O(n) ☑

You've hit the three goals for this lesson:

You can describe linear search: check items one by one until match/end.
You coded it and used it on a small unsorted list.
You know its worst-case time is O(n).
Before we wrap this lesson: want to do one quick, practical check — searching a list of gamer tags to see if a friend is in there — or are you ready to move on to the next lesson?

Try the gamer tag example

Nice, let's make it concrete 🎮

You have this unsorted list of gamer tags:

Python

```
gamers = ["ShadowFox", "NoScopeNate", "RonTheMage", "PixelPanda", "LagLord", "ClutchKitty"]
```

Your friend's tag is "RonTheMage".

We'll use linear search to find the index where your friend appears.

Step-by-step, the search will compare:

"ShadowFox" vs "RonTheMage" → no
"NoScopeNate" vs "RonTheMage" → no
"RonTheMage" vs "RonTheMage" → yes → stop, return this index
Your turn in the code editor:

Keep your linear_search(lst, target) function as you wrote it.
Create the gamers list above in your code.
Call linear_search(gamers, "RonTheMage") and print the result.

Run it, then hit Submit code, and I'll check that it finds your friend at the right index 👀

Python

```
def linear_search(lst, target):
    for index, item in enumerate(lst):
        if item == target:
            return index
    return -1

gamers = ["ShadowFox", "NoScopeNate", "RonTheMage", "PixelPanda", "LagLord", "ClutchKitty"]


result = linear_search(gamers, "RonTheMage")
print(result)
```

What do you think? 👇

In linear search, how does the algorithm examine the data to find a target value?

A.

It checks items one by one from the start until it finds the target or reaches the end.

B.

It jumps directly to the middle of the list and splits the search into two parts.

C.

It randomly checks positions in the list until the target is found.

D.

It sorts the list first and then searches only the first half.

Nice work, Ron 🎉
You nailed it.

Your answer is correct because linear search literally does:

check item 0, then item 1, then item 2, …
stop if it finds the target, or if it reaches the end with no match.

It doesn't jump around, doesn't sort, and doesn't skip parts of the list — just a simple one-by-one scan. That's why its worst-case time is O(n): in the worst case it has to look at every item.

You've:

Described linear search in your own words ☑
Implemented it in Python and used it on real examples ☑
Connected it to O(n) time ☑
Strong work on this lesson 👍
When you're ready, go ahead and jump into the next one in the Learn tab.

```
def linear_search(lst, target):
    for index, item in enumerate(lst):
        if item == target:
            return index
    return -1


my_list = [2, 4, 5, 6, 8, 10, 12, 15, 16]

result = linear_search(my_list, 2)
print(result)

def linear_search(lst, target):
    for index, item in enumerate(lst):
        if item == target:
            return index
    return -1

gamers = ["ShadowFox", "NoScopeNate", "RonTheMage", "PixelPanda", "LagLord", "ClutchKitty"]


result = linear_search(gamers, "RonTheMage")
print(result)
```